Deliverable 3: controlled evolution and contracts

Individual · 15% of course · Due November 30, 2026, 23:59 Toronto time

This brief announces the full scope in advance. The change becomes the active assessed requirement on **November 9**. No new requirement is introduced in the November 30 lecture. Continue the D2 codebase and preserve the D1/D2 behaviours that the change does not deliberately alter.

The change request: policy version 2

Activate the accessible-space rule: when `requiresAccessible` is true, a target must be accessible in addition to being open, unoccupied and different from the current space. Increment the policy version from 1 to 2. A proposal created under version 1 must not execute under version 2, even if its target would otherwise still be eligible. Require a fresh investigation and proposal.

For the supplied B1 fixture, version 2 alternatives are exactly C03. B12 is no longer eligible. A booking that does not require an accessible space may still use either available target. The model must receive the updated allowed candidates; Java still enforces the rule if the model proposes an ineligible target.

This is a deliberate change to a synthetic baseline, not advice about real accessibility policy. Do not remove general availability checks while adding the new condition.

Required work

D3.1 — Implement and demonstrate the change. Show the baseline and version-2 outcomes on clean fixtures. Demonstrate rejection of an old approved proposal after the policy change and successful execution of a fresh valid proposal. Preserve one-commit behaviour. Show what the periodic checker does when the policy version changes.

D3.2 — Refactor a demonstrated problem. Identify a concrete smell, duplication or dependency problem in your D2 code. Make a small coherent refactoring, preserving unaffected behaviour. Provide the before/after source locations, reason, regression command and result. Do not create an artificial bad implementation solely to remove it or claim that reformatting is a structural refactoring.

D3.3 — Static analysis. Run the compiler's `-Xlint:all` over your own sources, record the command and classify any warnings. Fix relevant warnings or justify a narrowly scoped remaining warning. Include one manual dependency/encapsulation review finding that the compiler does not establish. A zero-warning build is not proof of correct agent behaviour. Optional additional tools may be used but are not required for marks.

D3.4 — Contracts. Select two small deterministic operations from your implementation. Provide JML preconditions, postconditions and frame conditions, plus at least one meaningful class invariant and one loop invariant/variant. Suitable operations include attempt consumption, counting or searching candidate data, and a small controlled state update. State assumptions about referenced objects. Do not claim that a local JML proof verifies the model or the entire workflow.

Run the supported OpenJML setup on those selected files when available. Include exact command, tool version, result and unresolved obligations. If the course has not supplied the supported OpenJML environment, submit the compilable annotated source, a manual proof explanation and ordinary boundary tests under the announced tooling contingency; do not invent verifier output. The TA/instructor must provide and pilot the formal tool before it becomes an execution requirement.

D3.5 — Final explanation. Update the class, sequence and state diagrams. In at most six pages plus diagrams, trace one accepted operation and one rejected operation; explain the change's impact on responsibilities; discuss two design alternatives, one residual limitation and how your tests/checks constrain

the conclusion. Preserve the distinction between Java execution, Jupiter testing, static warnings, formal verification and a live model run.

Required regression and change cases

Case	Initial/input	Expected observation
D3-A	Policy 1, B1 fixture	B12 and C03 eligible
D3-B	Activate policy 2	Only C03 eligible for B1
D3-C	Version-1 approved proposal, then policy 2	STALE; no domain mutation
D3-D	Fresh version-2 proposal to B12	Rejected; no commit
D3-E	Fresh version-2 proposal to C03, approve	One commit to C03
D3-F	Repeat D3-E execution	No second commit/notification
D3-G	Non-accessibility-requiring booking	General available alternatives remain eligible
D3-H	Model ignores policy and proposes B12	Application rejects despite model output
D3-I	Periodic tick after policy-version change	New work can be emitted once for the new version pair
D3-J	Contracted attempt budget reaches limit	Remaining nonnegative; next consume fails/has prohibited pre-state as specified

Rubric: 100 release points

Criterion	Points	Full-credit expectation
Correct policy evolution and stale-proposal handling	25	Required change cases, fresh investigation and preserved controls
Regression tests and traceable outcomes	20	Unaffected tests retained; new negative/boundary cases; truthful outputs
Refactoring and static-analysis interpretation	15	Demonstrated structural improvement and reasoned warning/dependency review
Contracts, invariant/variant and verification reasoning	20	Precise specifications, frame, correct loop reasoning and honest tool status
Architecture and updated diagrams	10	Code/diagram consistency and explained impact of change
Reproducibility, reflection and attribution	10	Complete submission, executable commands and specific limitations

There is no minimum source-line count, no reward for adding uncontrolled tools, and no required frontend. A smaller coherent system satisfying all core behaviours can receive full marks.

Submission and individual authorship

This is an individual project. Submit your own repository snapshot and explain every included design decision. Do not submit a shared implementation, team report or another student's source. Discussion of general concepts is permitted; exchange of assessed code, diagrams or solutions is not. Designated AI assistance is permitted with the disclosure below. Supplied infrastructure is identified explicitly and may be retained with attribution.

Submit one ZIP through the named eClass assignment before **23:59 Toronto time** on the stated date. Name it `EECS3311_Dn_STUDENTNUMBER.zip`, replacing `Dn` and `STUDENTNUMBER`. Its single top-level folder

must contain `src/`, `test/`, `run.sh`, `README.md`, `DESIGN.pdf`, `AI_USE.md`, `TEST_PLAN.md`, `CHANGELOG.md` and `results/`. Keep generated `.class` files, IDE caches, model weights, secrets and downloaded libraries out. A private Git repository is recommended for development; the submitted ZIP is the marking snapshot. Do not rely on a remote branch changing after the deadline. Keep a copy of the uploaded receipt and ZIP.

`README.md` must state the JDK, exact execution commands, supported scenario(s), known limitations and any permitted starter files reused. Both `bash run.sh check` and the appropriate prepared demonstration must run from the extracted project root. Jupiter tests use `bash setup-junit.sh` once on the supported network, then `bash run.sh test`; the TA will provide/cache the pinned dependency. Include genuine command output with date, command and environment in `results/`. Do not label a plain-Java check run as a JUnit run or label replay as a live-model run.

In `AI_USE.md`, identify each tool/model used, date or version when known, the task it helped with, the source files or diagrams affected, what you changed or rejected, and how you checked the result. State "No generative AI assistance used" when applicable. Whole personal chat histories are not required. Remove credentials and personal information from any excerpt. Code produced with assistance remains your responsibility. The instructor may ask you to explain or make a small change to your submitted work under the course's academic-integrity procedures; there is no separate undisclosed viva mark.

Submit even if one part is incomplete: identify the affected requirement precisely. Do not hide a failure by deleting the test or editing a captured result. Required extensions cannot be replaced by a polished user interface. Optional features earn no substitute marks for missing core requirements. Approved accommodations and extensions take precedence over ordinary deadlines; request assistance through eClass promptly. Late/absence arrangements are governed by the published syllabus and approved course procedures, not by this brief inventing a separate penalty.